



IntVariable

🕒 Created time	June 20, 2026 1:06 PM
☰ Category	CommonVariables
👤 Last edited by	 R.A R.A
🕒 Last updated time	July 3, 2026 3:11 AM

```
//Yes: 2, 18, 2854, -9

//No: 1.2, -5.7

int age = 0;

age = 43;

int ageInTenYears = age + 10;

//Two billion +/-
//signed Int32
//unsigned - 4 Billion
//bit - 0 or 1
//byte - 1 byte or 8 bits - 00000000
//1, 11, 111

Console.WriteLine(age);
Console.WriteLine(ageInTenYears);

//don't divide ints - Console.WriteLine(age / 2);
```

מכיל מספרים שלמים.

אנחנו יכולים לבצע עליו מניפולציות כמו חיבור, חיסור וכפל שעובדים מצוין עם מספרים שלמים **Integers**.

~~~~~ Type Int

כדי להצהיר על משתנה, אנחנו מצהירים על הסוג שלו.

Int = Integer

כלומר מספר שלם, אינטג'רים מכילים מספרים שלמים.

מספרים שלמים הם כל המספרים שאין בהם נקודה עשרונית:

מספרים שלמים: 2, 18, 2854, -9

לא מספרים שלמים: 1.2, -5.7

שברים עשרוניים לא יעבדו בתוך מספרים שלמים Integers.

הערך ההתחלתי (Initial value) של Int, במידה ולא נציב בו ערך אחר, הוא 0.

משתנה מסוג Int

- **הצהרת המשתנה:** שפת #C היא שפה מוקפדת, זה אומר שכדי ליצור משתנה, חובה קודם כל להגדיר למערכת איזה סוג מידע הוא יחזיק.

השימוש במילה השמורה **int** מודיע למערכת להקצות זיכרון ספציפית למספר שלם.

- **אופי המידע (Integer):** המשתנה הזה מקבל רק מספרים "עגולים", בין אם הם חיוביים (2854) ובין אם הם שליליים (-9). הניסיון להכניס לתוכו מספר עשרוני (כמו 1.2) פשוט לא יעבוד – הקומפיילר יזהה חוסר התאמה בסוג הנתונים ויזרוק שגיאה.

- **פעולות מתמטיות (Manipulation):** הטיפוס **int** מיועד ומותאם לעבודה מהירה וחלקה עם פעולות אריתמטיות בסיסיות (חיבור, חיסור, כפל, וחלוקה של שלמים).

- **ערך ברירת מחדל:** בעולם הפיתוח קוראים לזה דיפולט.

אם רק נכריז על המשתנה אבל לא נכניס אליו שום מספר מפורש, סביבת ה-.NET לא תשאיר אותו ריק, אלא תציב בתוכו אוטומטית את הערך **0** כדי למנוע שגיאות של זיכרון לא מוגדר.

~~~~~

```
//age = "43";  
Console.WriteLine(age);  
age = 43;
```

- אנחנו לא משתמשים במירכאות "" סביב המספר 43, מכיוון שזאת אינה מחרוזת String:

```
age = 43;
```

- אם היינו שמים מרכאות "" סביב המספר:

```
age = "43";
```

- היינו מקבלים שגיאה, מכיוון שאי אפשר להכניס String לתוך משתנה מסוג מספר שלם (Int), סוגי המשתנים לא מתערבבים.

- אנחנו יכולים להדפיס את זה לקונסול:

```
Console.WriteLine(age);
```

- **המשמעות של מרכאות כפולות ""** : ב-C#, כל דבר שמוקף במרכאות כפולות מזוהה אוטומטית כטקסט בלבד (String).  
כשאנו כותבים **"43"**, המחשב לא רואה את זה בתור ערך מתמטי של ארבעים ושלוש, אלא בתור התו '4' והתו '3' שמחוברים יחד כטקסט.
- **הפרדה קשיחה בין סוגי נתונים**: #C היא שפה מאוד קפדנית, סוגים לא מתערבבים, אם קבענו שמשתנה מסוים יכול Int, אי אפשר פתאום להכניס אליו String.  
הקומפילר של #C חוסם את זה מיד וזורק שגיאה, כדי למנוע מצב שבו בהמשך הקוד ננסה לבצע פעולת חיבור או כפל על טקסט.
- **הדפסה נכונה למסך**: שכשאנו קוראים לפקודה `Console.WriteLine(age)`, אנו מעבירים לה את המשתנה `age` (בלי מרכאות סביב המילה `age`).  
הפקודה יודעת לגשת לזיכרון, לקחת את המספר 43 ששמור שם, ולהציג אותו על המסך.

~~~~~

```
Console.WriteLine(age.ToString());
```

- **המרת משתנה מסוג שלם Int למחרוזת String**:

```
Console.WriteLine(age.ToString());
```

- פעולה זו לוקחת את הערך שנמצא בתוך משתנה ה-Int שנקרא `age`, וממירה אותו ל-String.
- במקרה זה, הפקודה `Console.WriteLine(age.ToString())` לוקחת את המספר 43 ועוטפת אותו במרכאות (כלומר מתייחסת אליו כ- **"43"**).
- **פעולה ToString()** : ב-C#, לכל משתנה יש סוג נתונים ספציפי ואי אפשר להתייחס למספר מתמטי (Int) בדיוק באותו אופן שבו מתייחסים ל-String.
הפקודה `ToString()` היא מתודה מובנית וסופר נפוצה שתפקידה לתרגם נתון מסוים לפורמט של טקסט קריא.
- **הלוגיקה מאחורי הקלעים**: כאשר מריצים את הקוד, המערכת ניגשת למשתנה `age` (שמחזיק נתון מתמטי 43). פעולת ה- `ToString()` לא הורסת או משנה את המשתנה המקורי, אלא יוצרת **עותק חדש** שמיוצג כטקסט. המרכאות "" מסמלות למחשב שעכשיו מדובר בתווים, כלומר, זה כבר לא ארבעים ושלוש אלא התו '4' ואחריו התו '3'.
- **המשמעות בפועל**: ברגע שהפכנו את ה-Int ל-String, הוא מקבל תכונות של מילה.
כבר אי אפשר לבצע עליו פעולות חשבון (כמו לחבר או לכפול אותו), אבל הפעולה הזו הכרחית כשרוצים להציג את הנתון למשתמש (למשל על ה-Console, בתוך ממשק צד לקוח, או כחלק ממשפט כמו "הגיל שלך הוא 43").

~~~~~

**עד כמה גדול ה-Int הזה יכול להיות:** קצת יותר משני מיליארד +/- (כלומר 2 מיליארד חיובי ו-2 מיליארד שלילי).  
**טכנית, מאחורי הקלעים זה נקרא:** signed Int32 (שלם מסומן, 32 ביט).

## מסומן signed

יש סימן פלוס או סימן מינוס (אנחנו לא מציגים את סימן הפלוס רק בגלל שכולנו יודעים שאם אין סימן, המשמעות היא שהמספר חיובי), אבל ביסודו של דבר יש לו פלוס או מינוס.

**אם יש "signed", יש גם "unsigned" (לא מסומן):** המשמעות היא שהוא יכול להכיל 4 מיליארד (רק חיוביים).

**בדרך כלל אנחנו משתמשים ב-signed Int32:** מכיוון שהוא מאחסן 2 מיליארד לכיוון החיובי ו-2 מיליארד לכיוון השלילי, וזה כל מה שאנחנו צריכים.

בסביבת הפיתוח המודרנית של C#.NET, ניהול זיכרון הוא קריטי.

## מה הוא Int32

המילה int ב-C# היא בעצם קיצור (Alias) למבנה נתונים שנקרא מאחורי הקלעים Int32.  
המספר 32 מייצג את כמות הזיכרון שהמחשב מקצה למשתנה הזה – 32 ביטים (Bits) של מידע.

## המשמעות של Signed (עם סימן):

כברירת מחדל, int הוא "Signed".

זה אומר שהמחשב לוקח ביט אחד מתוך ה-32 כדי לזכור את הסימן של המספר (פלוס או מינוס).

נשארים לנו 31 ביטים כדי לאחסן את המספר עצמו.

הנוסחה המתמטית לגבול העליון היא  $2^{31} - 1$ , שזה בדיוק 2,147,483,647 (קצת יותר מ-2 מיליארד).

לכן, הטווח הוא בערך מינוס 2 מיליארד עד פלוס 2 מיליארד.

## המשמעות של Unsigned (ללא סימן):

אם אנחנו יודעים שמשתנה מסוים בחיים לא יהיה שלילי (למשל, גיל של אדם), אפשר להשתמש בטיפוס הנתונים uint (Unsigned Int).

במצב כזה, המחשב לא מבזבז ביט על שמירת מינוס או פלוס, ומשתמש בכל ה-32 ביטים עבור המספר עצמו.

הנוסחה הופכת ל- $2^{32} - 1$ , מה שמכפיל את הטווח המקסימלי ל-4,294,967,295 (קצת יותר מ-4 מיליארד), אבל כאמור, חיובי בלבד.

## מטרת השימוש ב-Signed כברירת מחדל

זוהי פרקטיקת העבודה הסטנדרטית בתעשייה.

טווח של 2 מיליארד לכל כיוון מכסה 99% מהצרכים היומיומיים בפיתוח מערכות.

שימוש ב-int רגיל מונע באגים לא צפויים שעלולים לקרות כשמנסים לחסר מספרים ב-uint ופתאום התוצאה צריכה לרדת מתחת לאפס, מה שיגרום לקריסה או תוצאה שגויה.

~~~~~

ה-Bit הוא 0 או 1.

זה מה שמחשבים משתמשים בו, בעצם זה שער, האם השער פתוח או סגור, זה כל מה שזה.

אז יש לנו את הערכים של פתוח וסגור, פתוח זה 0, סגור זה 1.

זהו בסיס בינארי, ואנחנו לא בהכרח מתעסקים עם בינארי ב-C# כל כך הרבה, למעשה כמעט אף פעם לא.

אבל על המערכת הבינארית הזו מבוססים הנתונים ואיך שהם עובדים, אז אנחנו נראה את המושג של ביטים ובייטים.

ה-bit אחד הוא 0 או 1.

ה-byte אחד הוא 8 bits, ידוע גם כ-00000000 (8 אפסים), וכל אחד מהאפסים האלה יכול להיות 1.

ספירה בבינארית: חשוב מאוד להבין את זה בשלב זה, ולמרות שזה עשוי להיראות קצת תיאורטי זה לחלוטין לא. מפתחים חדשים מתבלבלים מזה ומקבלים החלטות על סמך הבלבול שלהם.

1 bit

0 מייצג 0, ו-1 מייצג את המספר 1.

2 bits

00 מייצג 0, 01 מייצג 1, 10 מייצג 2, 11 הוא 3.

ככה בינארי סופר.

בעזרת 11 אפשר לייצג 0, 1, 2, 3, שזה 4 מספרים.

עם bit אחד יכולנו לייצג 2 מספרים (0 או 1), עם שני bits שזה (11) אנחנו יכולים לייצג כמות כפולה של מספרים בעזרת תוספת של bit אחד בלבד.

3 bits (000)

יכולים לייצג 0 (000), 1 (001), 2 (010), 3 (011), 4 (100), 5 (101), 6 (110) ו-7 (111).

אז אנחנו יכולים לייצג מ-0 עד 7, שזה 8 מספרים בסך הכל.

אז 3 ה-bits יכולים לייצג 8 מספרים, 2 ה-bits יכולים לייצג 4 מספרים, ו-bit ה-1 יכול לייצג 2.

שימו לב שהכפלנו את הכמות בכל פעם שהוספנו עוד bit אחד.

כשאנחנו רואים 00000000 (8 ה-bits) זו הכפלה בכל פעם שאנחנו מוסיפים 0.

כשאנחנו אומרים 16 ה-bits, זה לא גדול פי שניים מ-8 ה-bits, וזה הבלבול שכולם נתקעים עליו.

9 ה-bits זה כפול בגודל מ-8 ה-bits.

10 ה-bits זה פי 4 בגודל מ-8 ה-bits, וזה ממשיך לעלות בהכפלה.

כשאנחנו רואים Int32 (כלומר 32 ה-bits) זה אומר שיש 32 אפסים, שזה מספר המקומות שאנחנו יכולים לאחסן בהם מספרים, וזה מספר עצום.

אם אנחנו רוצים ללכת על גדול יותר, האופציה היא Int64.

זה מספר כל כך גדול שקשה לתפוס אותו, ואנחנו לא משתמשים בו.

אחת הטעויות ההגיוניות הנפוצות ביותר בתכנון מערכות, היא שהרבה פעמים כשבחרים סוג משתנה לשמירת מספרים, קל לחשוב שמשתנה מסוג Int64 פשוט יכיל פי שניים יותר מספרים ממשתנה מסוג Int32, פשוט בגלל ש-64 זה כפול מ-32.

ההיגיון הזה שגוי לחלוטין, מכיוון שמחשבים עובדים בשיטה בינארית, כל מקום (ה-bits) שמוסיפים למשתנה לא מגדיל את הקיבולת בקצת, אלא מכפיל את סך כל האפשרויות פי שניים! המשמעות היא שאם יש משתנה של 8 ה-bits ועוברים ל-9 ה-bits, כבר הכפלנו את הגודל המקסימלי פי שניים. אם נעבור ל-10 ה-bits, הכפלנו פי ארבעה.

המעבר בין Int32 ל-Int64

המעבר הוא לא הוספה של ה-bit אחד או שניים, אלא הוספה של 32 ה-bits חדשים. זה אומר שהכפלנו את הגודל פי 2, ואז שוב פי 2, וכך הלאה 32 פעמים. התוצאה היא ש-Int64 לא גדול פי שניים מ-Int32, אלא גדול ממנו פי יותר מארבעה מיליארד! בסביבות פיתוח מתקדמות ב-2026, כשאנחנו בונים ארכיטקטורת Full Stack ומתממשקים מול בסיסי נתונים ו-SQL, אנחנו כמעט תמיד נשתמש ב-Int32 (שנקרא ב-C# בפשטות int) עבור מספרים שלמים. הוא מספק מקום ליותר משני מיליארד ערכים. שימוש ב-Int64 (שנקרא C# long) שמור למקרים קיצוניים בלבד, כמו מפתחות זיהוי (IDs) כבדים במיוחד. הבנת המנגנון הזה תעזור לכתוב קוד יעיל יותר שמקצה נכון את הזיכרון ולא מעמיס על המערכת לחינם.

~~~~~

אנחנו רוצים לדעת מה יהיה הגיל שלנו בעוד 10 שנים:

```
int ageInTenYears = age + 10;
Console.WriteLine(ageInTenYears);
```

כך אנחנו עובדים עם מספרים שלמים (Integers), אנחנו יכולים לחבר, לחסר, להכפיל ולחלק אותם. אנחנו לא משנים את המספר המקורי בסוג כזה של משוואה:

```
int ageInTenYears = age + 10;
```

אם אנחנו רוצים לשנות את המספר המקורי במשוואה כזו:

```
age = age + 10;
```

זה ישנה את המשתנה המקורי, על ידי לקיחת ערך התוצאה והכנסתו חזרה למשתנה המקורי, מה שישנה את הגיל ל-53.

אנחנו צריכים להיזהר עם חילוק מכיוון שאם נכתוב את זה כך:

```
Console.WriteLine(age / 2);
```

התוצאה תהיה שגויה, התוצאה תהיה מספר שלם כי אנחנו משתמשים ב-Int.

חיבור, חיסור וכפל עובדים נהדר עם מספרים שלמים.  
אנחנו צריכים להיות מאוד זהירים עם חילוק כי התוצאות יהיו שגויות.

## שני עקרונות ליבה קריטיים בעבודה עם טיפוס הנתונים Int שמכיל מספרים שלמים בלבד.

**העיקרון הראשון** מדבר על ההבדל בין שמירת תוצאה במשתנה חדש לבין עדכון משתנה קיים. כשיוצרים משוואה ומציבים אותה לתוך משתנה חדש, המשתנה המקורי ששימש לחישוב נשאר בדיוק כפי שהיה. אבל, אם רוצים שהמשתנה המקורי עצמו יגדל או יקטן, חייבים לבצע השמה מחדש לתוכו. מבחינה לוגית מאחורי הקלעים, התוכנית קודם כל מחשבת את כל מה שנמצא בצד ימין של סימן השווה, ורק אז לוקחת את התוצאה הסופית ושומרת אותה בתוך המשתנה שבצד שמאל. הפעולה הזו למעשה דורסת את הערך הישן.

**העיקרון השני** מזהיר מפני מלכודת החילוק של מספרים שלמים. טיפוס Int לא מתוכנן בשום צורה להחזיק שברים. המשמעות היא שאם נבצע פעולת חילוק שהתוצאה שלה אמורה לכלול נקודה עשרונית, המערכת לא תעגל את המספר כלפי מעלה או מטה.

היא פשוט תחתוך את החלק העשרוני ותזרוק אותו לפח. לדוגמה, אם הגיל הוא 53 ונחלק אותו ב-2, התוצאה האמיתית היא 26.5. אבל בגלל שזה Int, נקבל פשוט 26, מה שיוביל לחישוב שגוי ואובדן מידע. לכן, חיבור, חיסור וכפל הן פעולות בטוחות וישירות ב-Int, אך בחילוק חובה לשים לב לזה מראש.